

Four Measurement Failures in a Small-Scale Language Model Experiment

Abstract

A common argument for filtering fact-heavy pages out of language model pretraining data is that memorising facts consumes parameters the model could otherwise spend on reasoning. We set out to test this claim directly at small scale, and over four rounds of experiments we obtained four null results. Each was uninterpretable, because in every round a different component of our measurement pipeline had failed in a way our existing checks did not detect: a decoder that scored a working benchmark at 4.7% when its true score was 93.8%, a data pipeline that replaced most of one training stream with another, a control condition that removed less learnable signal than the treatment it was matched against, and a knowledge probe that could not distinguish a fact the model had memorised from one we invented. In this paper we report each failure, quantify it, and give a cheap test that detects it. We also report the one measurement that survived. Under teacher forcing, loss at fact positions is the same for people the model trained on as for people it never saw, differing by -0.0010 ± 0.0132 nats. The model has fitted the biography format and the frequencies of the value pools, and we find no evidence that it stored anything specific to individual people. We first reached that conclusion with a broken probe, and only later built an instrument that can detect storage when storage is present.

1 Introduction

Language models are increasingly trained on curated rather than scraped data, and one recurring curation decision is how much factual content to include. Several authors have suggested that arbitrary facts are a poor use of a small model’s parameters, on the grounds that memorising them consumes capacity that would otherwise support reasoning. If this is true, then removing or hiding factual content during training should improve reasoning performance at fixed model size, and the effect should grow with the amount of factual content removed.

We attempted to measure this effect directly. Working at 40M parameters, where a full training run costs about a day on one GPU, we ran four rounds of the experiment. Each returned a null result: no detectable difference between the treatment condition and its control. Because null effects are plausible at this scale, we did not initially treat repeated nulls as evidence that the pipeline itself had failed.

Each null was in fact uninterpretable, for a different reason each time. This paper reports those four failures rather than the hypothesis, which we still regard as untested. For each we give the magnitude and a test that would have caught it; none of

the tests requires a GPU and the most expensive takes fifteen minutes on a laptop. Three of the failures are properties of tooling that many groups write for themselves. The fourth is of a kind the usual defences do not catch: that measurement had a control attached, the control passed, and the measurement was still worthless, because nothing had shown it could return a positive under any circumstances.

Section 2 describes the model, the data, and the metrics. Sections 3 through 6 report the four failures in the order we found them. Section 7 discusses what they have in common, and Section 8 states the limits of the evidence.

2 Background and Setup

Model and data. We trained a 40.6M-parameter decoder-only transformer from scratch. The training corpus is synthetic, which lets us control exactly how much factual content the model sees. It interleaves four streams of text. The first is short biographies of invented people, each stating six attributes such as a birth date and an employer, with values drawn from fixed pools. These are the “facts” whose effect we wanted to measure. The second is modular-arithmetic word problems accompanied by step-by-step worked solutions, following the design of Ye et al. [2]. The third is chains of logical deduction. The fourth is ordinary natural text, included as filler so that the total size of the corpus can be held fixed while the proportions of the other three vary.

The main run described here trained for 31,280 optimisation steps on 16.4 billion tokens, and its corpus contains 1,531,800 invented people, each appearing 100 times.

Reporting accuracy. All accuracies below are *closed-book exact match*: the model sees a problem with no reference material and its answer counts as correct only if it matches the target string exactly. We compare against the *majority-class baseline*, the accuracy obtained by always giving whichever answer is most common in the test set. Our arithmetic answers are remainders after division by a small integer, so some occur far more often than others, and comparing against uniform chance would flatter the model.

Measuring loss. Losses are in *nats*, the natural-logarithm form of cross-entropy. A loss of $\ln k$ nats means the model is as uncertain as if choosing uniformly among k options, so lower is better and a change of 0.69 nats is a factor of two in the probability assigned to the correct token.

Storage capacity. Allen-Zhu and Li [1] estimate how much factual knowledge a transformer can hold, finding a ceiling

near two bits per parameter when each fact appears about a thousand times in training, and near one bit at a hundred appearances. Our model sees each fact 100 times, and its corpus would need about two bits per parameter to store completely, so the corpus asks for roughly twice what the model can hold. Our operating point coincides with one of their reported measurements rather than falling between them, so this ratio does not depend on interpolating their curve, and their hundred-appearance figure would have to double before our corpus stopped exceeding capacity.

3 A Decoder That Attended to Its Own Padding

To evaluate the model we generate answers for many problems at once. Prompts have different lengths, so batching them requires padding to a common length. Our generator padded on the left, used the end-of-text token as the padding symbol, and supplied no *attention mask* — the array telling a transformer which positions to ignore. Without one the model attends to padding as though it were content. A comment in our code justified this on the grounds that the pads were “ordinary end-of-text tokens the model has seen as separators,” which is exactly the problem. During training that token marks a boundary between documents, so a prompt preceded by a run of them looks like the opening of a new document, and the model does what it was trained to do there: it writes one. The output is fluent, correctly formatted, and parses cleanly as an answer, which is why the failure survived our output checks. The model was answering a question it had invented.

Table 1 shows the effect on a single checkpoint, evaluated three ways on the same 64 held-out problems.

decoding method	exact match
batched, padded to batch maximum	3/64
one prompt at a time	60/64
batched, after the fix	60/64

Table 1: The same weights on the same items, decoded three ways.

A replication at a different random seed gave 0/64, 63/64 and 63/64, and every repaired batched generation was character-for-character identical to its unbatched counterpart. By varying the amount of padding on a fixed prompt we found that corruption sets in somewhere between 16 and 24 padding tokens.

The defect also explains two anomalies we had attributed to the model itself. Measured accuracy had been *rising* with the number of arithmetic steps a problem required, the opposite of the expected pattern, which we had read as a model emitting a near-constant answer. In fact padding fills the gap between a prompt and the longest prompt in its batch, and harder problems have longer prompts, so the easiest problems received the most padding and were damaged most. Separately, 47% of generations had fallen outside the set of syntactically valid answers, which we had taken as evidence that the model could not produce a well-formed response.

The general fix is to supply an attention mask, which costs nothing. The test is one comparison: decode a prompt on its

own, decode it again inside a batch of mixed-length prompts, and require identical output.

With the decoder repaired the benchmark works. Table 2 reports the finished run on 1,500 problems from the difficulty range it trained on and 750 harder ones. Aggregate accuracy in the trained range is 96.0%, against a majority-class baseline of 7.5%, and the deduction task reaches 0.950 against a baseline of exactly 0.500. One to three operations are saturated and no longer informative; four still has room, and the sharpest remaining signal is the fall from five operations to six.

operations	in training range	<i>n</i>	exact match
1	yes	399	99.7%
2	yes	384	99.2%
3	yes	375	95.7%
4	yes	342	88.3%
5	no	183	41.5%
6	no	194	7.2%
7	no	190	5.3%
8	no	183	4.9%

Table 2: Final checkpoint, by problem difficulty.

4 A Control Condition With Incompatible Criteria

The experiment compares two training conditions. In the treatment, the loss is not computed on the tokens that spell out a fact’s value, so the model is not asked to predict them. In the control, the same number of tokens is excluded from the loss, but they are ordinary text rather than fact values. The point of the control is to separate the effect of *which* tokens were excluded from the effect of simply excluding some.

For the comparison to isolate the first effect, the excluded spans in the two conditions should match on everything except their content. Our match constrained four properties: how many tokens are excluded, how long each contiguous excluded span is, where in the document those spans sit, and how difficult the excluded tokens are to predict. The last is the important one, since excluding easy tokens removes less learnable signal than excluding hard ones.

Measured across the corpus, the match failed. Treatment spans averaged 1.9598 nats and control spans 1.2712, a relative shortfall of 35.1% against a tolerance of 20% that we had fixed before running the experiment. Our first interpretation was that the corpus simply contains no ordinary text difficult enough to match fact values. Table 3 shows that this is wrong, and the way it is wrong is more useful than the original observation. Relaxing the matching criteria one at a time shows that sufficiently difficult tokens do exist, but only when the requirement that they form contiguous spans is dropped.

The reason this generalises is that facts and ordinary prose distribute their surprising tokens differently. A fact value is a contiguous run of hard-to-predict tokens; comparably hard tokens in ordinary text occur singly and scattered. A control can therefore reproduce the shape of what the treatment removes, or the quantity of learnable signal it removes, but not both at once. We do not think this is specific to our corpus, and we suggest that work using matched controls of this kind

control construction	mean nats	shortfall
as shipped: length + position	1.2712	35.1%
best possible, length only	1.5667	20.0%
scattered, difficulty-matched	1.7677	9.8%
scattered, hardest available	1.8644	4.9%

Table 3: Pricing each matching criterion separately, over 800 documents.

state which of the two properties it chose to preserve. We had not realised we were choosing.

5 A Data Stream That Silently Became Another

Our corpus builder mixes the four streams according to requested proportions. The fact stream is finite: with a fixed number of invented people, each appearing a fixed number of times, it can emit only so many documents. When the requested proportion demanded more than that, the builder made up the shortfall from the filler text so that the total token count remained exact. Holding the token count fixed is a requirement of the design, because every experimental condition must train for the same number of steps.

This reallocation is the correct behaviour and it produced no warning. Table 4 shows what one pilot corpus was asked for and what it actually contained.

stream	requested	realised
facts	50%	3.75%
filler text	10%	56.25%

Table 4: A corpus built for an experiment about fact load.

The pilot that produced our first decision to stop the experiment had therefore trained on a corpus that was 3.75% facts. It passed every integrity check we had, because each check tested the corpus for internal consistency — that the token counts add up, that the masks align with the text — rather than testing it against its own specification. The realised proportions were written to a metadata field that nobody read.

The remedy is to record the requested proportions next to the realised ones and to fail the build on any discrepancy. The requested values have to be stored separately from the working budget, because the reallocation overwrites the budget it draws from.

6 A Knowledge Probe That Could Not Detect Knowledge

The hypothesis under test assumes the model stores the facts it is shown, so we checked. Loss at fact positions in training documents is 1.9732 nats, against 25.03 on an earlier corpus where facts appeared too rarely to be learned. That improvement is real but smaller than it looks: 1.9732 nats per token works out to 53.08 bits per person, against 52.96 bits for a predictor that knows only how often each value occurs in the corpus. The model has reached the value-pool frequencies and no further.

To ask whether it had nonetheless stored the individual associations, we queried it with a phrasing that never appears

in training and converted its answers into *recoverable bits*: how much better it predicts the true value than that frequency baseline would. It scored well below zero, and so did a set of invented people it had never seen. We wrote this up as a model that had stored nothing retrievable.

That description is correct, as we show below, but the evidence we had for it was worthless. We discovered this by asking the probe for a result that had to come back positive. If a probe can detect stored knowledge at all, then it must separate people the model trained on from people we invented when the query uses a phrasing the model saw during training. Table 5 shows that it does not.

query phrasing	trained	invented	diff.
absent from training	−111.93	−112.10	+0.17
present in training	−67.83	−67.65	−0.18

Table 5: Recoverable bits per person. The probe cannot tell the two groups apart even where it must.

The arithmetic above points the same way. Training loss puts the model within 0.12 bits of the frequency baseline, whereas the probe reports it as 68 bits per person worse. Two measurements of the same quantity disagreed by a wide margin and we had quoted the one that suited our expectations.

A measurement that does not use a query. We therefore measured storage in the terms the training objective itself uses. We take a training document, present it to the model as training did, and read the loss only at the positions spelling out the facts. This is *teacher forcing*: the model receives the true preceding tokens rather than its own generated ones, so the loss measures prediction given correct context. If the model memorised a fact, its document must be cheaper at those positions, because that is where knowing the fact pays. A model that learned only which values are common gets no such discount.

people	nats/token	docs
trained, 100 appearances	1.9876 ± 0.0096	400
invented, never seen	1.9866 ± 0.0091	400
difference	-0.0010 ± 0.0132	$z = -0.08$

Table 6: Standard errors are computed over documents. Each person contributes two renderings, so clustering by person would widen them by up to a factor of $\sqrt{2}$; the bounds we quote below allow for that.

Validating the replacement. This measurement is subject to the same objection as the probe it replaces: a null result from an instrument that has never returned anything else is not evidence. We therefore constructed a case in which the answer must be positive. A small model trained for 250 steps on nothing but 48 biographies has certainly memorised them. On that model the same measurement separates the two groups by 1.4484 nats ($z = +18.7$), and training it further widens the gap rather than closing it, which establishes sensitivity to a large gap in this overfitting regime.

In the memorising case both groups draw their values from the same pools, so knowing which values are common cannot account for the separation. On the 40M model the same measurement finds no such difference, and after widening the interval for clustering it excludes any advantage larger than about 0.04 nats, roughly a fortieth of the gap it resolves on the memorising model.

We are careful about what this licenses. The toy run shows the instrument can detect entity-specific storage when that storage is forced into a small closed set. It does not show that sparse or partial storage across 1.5M people would appear as a teacher-forced gap of this size, and an average null can hide a memorised subset. The comparison also changes two things at once, since invented people differ from trained ones both in their attribute associations and in whether their names were seen at all. The cleaner test holds the people fixed and permutes their values among them, which we have not run. What we can say is narrower: under the training objective, at fact positions, people the model trained on are indistinguishable on average from people we invented, which is what one would expect if prediction there were driven by template and pool cues rather than by who the person is.

If the capacity estimate transfers to this setting, a load at twice that estimate predicts storage of about half the facts, whereas we measure a mean advantage indistinguishable from zero. Resolving that needs the capacity-matched run described below, so we report it as a discrepancy rather than as a result about capacity.

7 Discussion

The hypothesis we set out to test assumes that stored facts occupy capacity reasoning could otherwise use. In our setting we can detect no such stored facts, so there was nothing for the treatment to free and the experiment cannot speak to the hypothesis in either direction.

Three of the four failures are ordinary engineering defects. We report them because each had a cheap detector that we did not run, and because the tooling they occurred in is the kind most groups write for themselves. We do not claim they share a mechanism.

The fourth is different, and it is the one we think generalises. A negative control is the usual defence for a measurement like our knowledge probe, and we had one: the probe scored invented people no differently from real ones. But a broken probe and an empty model give identical readings under that control, so it cannot distinguish them. The control that separates them is the one that forces a positive result. We suggest the requirement is general: before a null result is allowed to bear on a hypothesis, the instrument that produced it should be shown capable of returning a positive on a case where the answer is known. Ours cost fifteen minutes of CPU time, against the four experiment generations it would have saved.

Whether the model stores facts when the corpus is sized to its capacity rather than twice over needs one more training run of about a day, and the corpus is built. Until then our null marks a failed precondition rather than evidence against the filtering argument.

8 Limitations

All model-dependent results come from one model size, one training run, and one random seed, on synthetic data. The difficulty measurements in Section 4 used an early checkpoint of the run later analysed, so the comparison between conditions is unbiased but the absolute values are not independent of the model. The storage result uses 200 people per group at one checkpoint. We use *stored* there in a narrow sense: that training on a person’s attribute lowers teacher-forced loss on that attribute’s true value. The measurement cannot exclude a representation that never surfaces in the training objective, and our validation establishes sensitivity to the first kind only. Because teacher forcing supplies the true prefix of a multi-token value, later tokens of a value can be predicted without knowing whose value it is, which makes the measurement conservative in a direction we have not quantified. We claim no novelty for the padding defect, a known failure mode of hand-written decoders that omit an attention mask, and we have not surveyed how often the other three occur elsewhere.

Appendix: Checks We Now Run

None of these requires a GPU.

1. Decode a prompt alone and inside a batch of mixed-length prompts, and require identical output.
2. Save generated text, not only parsed answers. Ours stored the parse and discarded the text, so four rounds of invented documents left no trace.
3. Report how hard the tokens a control removes are, not only how many, and price each matching criterion separately.
4. Record requested data proportions beside realised ones, kept separate from the budget the pipeline mutates.
5. Check benchmarks for saturation as well as for floor effects.
6. Before believing a null, make the instrument produce a positive. A negative control alone will not detect an insensitive instrument.
7. Reconcile two measurements of the same quantity numerically before trusting either.
8. Give a replacement instrument the validation you demanded of the one it replaced.

References

- [1] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of Language Models: Part 3.3, Knowledge Capacity Scaling Laws. *ICLR*, 2025. arXiv:2404.05405.
- [2] Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of Language Models: Part 2.1, Grade-School Math and the Hidden Reasoning Process. *ICLR*, 2025. arXiv:2407.20311.